Single Agent Systems & Agent Pipelines - Quiz

Instructions:
Answer the following questions. These questions are designed to test your understanding of agent systems, tools, and evaluation.

1. Explain the concept of a stateful directed graph in agent pipelines. How does it differ from a simple linear pipeline?

- A stateful directed graph is like a flowchart where each node is connected to the next one with the help of edges. Stateful means it remembers the previous decisions made from the earlier steps. An example of the stateful directed graph would be google maps as they pick your route based on your current location and it helps you in unfavorable situations like if you made a wrong turn, it would adjust or reroute instead of forgetting the current paths taken to reach your destination and starting afresh.
- Since it's a graph and not a straight line, it can take a different path if required which is branching and it can loop back or skip ahead in the same way maps do reroute you if there's traffic instead of forcing you down one fixed road.
- A simple linear pipeline is the opposite; it's like a basic to-do list so you follow from the top to the bottom one task after another with no memory of earlier tasks and no way to jump around or repeat a step if something goes wrong.

2. Describe the role of nodes and edges in an agent workflow. Give an example of each.

- Nodes, they are tasks or actions. Each node is a place where something specific happens like you use a calculate tool or do a certain task. In an agent workflow, a node is one unit of work such as the analysis of a query, calling a tool or generating the final response.
- Edges**,** they're used to connect the nodes. They don't have a certain functionality themselves and just decide which node comes next, so it carries the data towards another node and controls the flow like 'after analyzing the query, go to the text summarizer tool.'

3. What is conditional routing in an agent system? Design a simple rule-based routing logic for three different query types.

Conditional routing is basically following if this occurs then that happens system for queries. The agent looks at what the query is asking, checks it against some rules and sends it to a specific path based on what it finds rather than treating every query in the same way. A real-life example could be sorting mail at a post office, a letter doesn't go to a random shelf; Someone checks the zip code and routes it to the right truck based on that. Different zip codes lead to different routes. Query examples, in this you might want to pick three distinct, simple categories and a clear rule for each:

- If query contains math-related words like calculate, sum, average then ensure it is enroute to calculator tool
- If query contains the word keywords, then it goes to the Keyword Extraction Tool
- Anything that doesn't match a specific pattern should be enroute to a general response

4. Why are cycles (loops) important in agent pipelines? Provide a use case where a retry loop is necessary.

Cycles or loops are quite important in agent pipelines as not every task succeeds on the first go, so a pipeline that only moves forward has no way to recover from that, therefore, it fails and reaches a dead end. A loop lets the agent go back and try again instead of giving up right away. Without loops, the workflow would be one-shot meaning if it fails once, the whole thing breaks. An example of a retry loop is an API call that fails due to a temporary network issue or server timeout. Instead of the agent immediately throwing an error, a retry loop lets it try the call again, waits for a moment and re-attempts it. This may occur up to a few times before finally giving up and showing an error if it keeps failing.

5. Explain how a single-agent system can simulate multi-agent behavior internally.

- A single agent can enact like a multi-agent by switching between different roles depending on what step it's on.
- It does this by breaking the task into a couple of stages so instead of doing everything at once, it first analyzes the query, then, it decides which tool to use which is followed up by the generation of the final response.
- Each of these stages behave like their own mini agent with their own job even though it's the same system just running a different function at that point of time.
- Since it's one system, it can also loop back and revisit earlier stages if required like re-checking the query or retrying a tool call, which is an action a fixed multi-agent setup might not do as easily as a single agent might do.
- This approach keeps the system simpler to build and maintain since there's only one agent to manage but it still gets the flexibility of having different specialists handle different parts of the task.

6. What are JSON schema tools? How do they help in structuring tool inputs and outputs?

- JSON schema tools are like a rulebook for the data that goes between an agent and a tool, as it tells what fields are needed, what type they should be such as text or number and what format is expected.
- Due to this, the input gets checked before the tool even tries to use it, so if something doesn't match it gets caught early instead of breaking halfway through.
- It also keeps the output consistent every time, so the data doesn't come back random or messy as it follows the same structure.
- This basically helps different parts of the agent system understand each other properly since they're all expecting the same shape of data.
- Hence, using JSON schemas reduces errors and improves communication between different components of an agent system.

7. Compare sequential tool calls and parallel tool calls. When would you prefer one over the other?

Sequential tool calls happens when one task waits for the previous one to finish because it needs the result of the previous task. Parallel tool calls are for tasks that don't depend on each other at all or are entirely independent of one another, so they can run at the same time instead of waiting. Sequential is preferred when there's a real dependency, meaning the output of one step is literally the input the next step needs. Parallel is preferred when the tasks are independent, since running them together just saves time instead of waiting around for no reason.

The parallel calls have a faster response time and better efficiency as since you're not wasting time on stuff that didn't need waiting on. Sequential tool example, an agent first calls a search tool to find relevant documents, then passes those documents to a summarization tool to generate a summary. The summarization step cannot begin until the search results are available, so the calls must happen one after another. Parallel tool example, an agent needs to fetch the current weather and the latest stock price for a report. Since these two tasks don't rely

on each other's output, the agent can call both tools at the same time, reducing the total time needed to get the report ready.

8. How would you implement error handling in a tool-using agent? Provide at least two strategies.

As things can go wrong such as the failure of a tool or an API time out and we need to figure out what the agent does about it instead of just crashing or giving a useless response. Error handling is about how an agent responds when something fails during execution, and the goal is to keep the agent usable and reliable even when something goes wrong behind the scenes. Two strategies are:

1) Try-except blocks - this catches errors as they happen instead of letting the whole program crash. It is like a safety net, if something goes wrong inside the try block, the except block catches it and returns a meaningful error message instead of just breaking everything.
2) **Fallback responses –** rather than sending an error straight back at the user when a tool fails, the agent switches to a backup plan. Example, if a search tool is down, the agent could fall back to answering from its own general knowledge instead of giving up completely. This keeps the experience smoother since the user still gets something useful, even if it's not the ideal path.

These techniques improve reliability and help maintain a better user experience.

9. What is trajectory evaluation in agent systems? Why is it important beyond just checking final output?

Trajectory evaluation means looking at the entire path the agent took to get to that answer, not just the destination. That does include every decision it made, every tool it called and every intermediate step along the way. A relevant example of this would be an agent that's asked to pass a set of test cases. If you only check the final output, like whether the tests show an output as passed, the agent could take some problematic measures to get there, like deleting the test cases it keeps failing on or hardcoding values just to force a pass instead of actually solving the problem the right way. Looking at just the final output wouldn't catch this but trajectory evaluation would mainly because it looks at the actual steps and approaches the agent took, not just whether the end result looked correct. This leads to why it's important beyond just checking the final output:

- It catches the mistakes that the final answer hides, like an inefficient tool use or wrong reasoning that still landed on the correct result by chance.
- It helps with debugging, since you can see exactly where things went wrong if something did.
- It gives a much deeper understanding of how the agent actually behaves, which helps improve it over time rather than just patching the output.

10. Define task completion rate and cost metrics. How would you measure and optimize them in a real-world system?

Task completion rate refers to how often the agent actually succeeds at what it's asked to do. Out of all the tasks it attempts, what percentage does it actually finish correctly, and it's a simple ratio of successful tasks divided by total tasks attempted. Cost metrics refers to how much it costs the agent to get the job done which is not always in the terms of money. It could be the number of API calls made, how much processing time was used or actual dollar cost if you're paying per API call.

The way to measure these in a real system:

- Task completion rate - track the outcomes across many runs or tasks over time and calculate the percentage that succeeded.
- Cost metrics - log things like number of tool calls per task, response time per task and actual API costs per task.

How you can optimize them:

- Improve routing accuracy, so the agent sends queries to the right tool the first time instead of fumbling around or retrying unnecessarily.
- Reduce unnecessary tool calls, since every extra call adds to both cost and time.
- Use more efficient algorithms/approaches where possible, so the same task gets done with less resource use.

There's usually a tradeoff between the two as you want a high completion rate, so the agent actually gets the job done but you do want low cost so it's not burning through resources to get there as well. Balancing both of these is the actual goal when deploying something in the real world, rather than maximizing just one of them.